

Easy Tags

(V1.0)

An asset for Unity to extend the tag functionality

Mini tutorial and walkthrough

Author:
(FeroDev)

Table of Contents

Part 1: What is it?	3
Part 2: Walkthrough.....	3
Part 3: How to use it.....	4
3.1: Tag Manager Window	4
3.2: Generate Accessible Tags:	5
3.3: Multiple Tags Component	6
3.4: [Tag] Attribute and TagMask property	7
3.5: TagMask and Multiple Tags Example Usage	9
3.6: TagMaskUtils Class	10

Part 1: What is it?

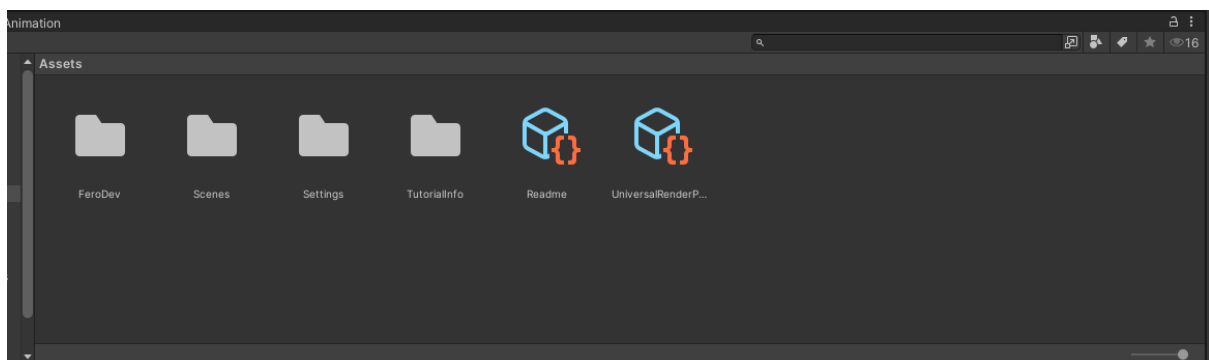
Easy Tags is designed to be an extension for the current tag system in Unity that adds more functionality, as well as to allow you a more user-friendly way of using tags in your games.

Following are the key features of Easy Tags:

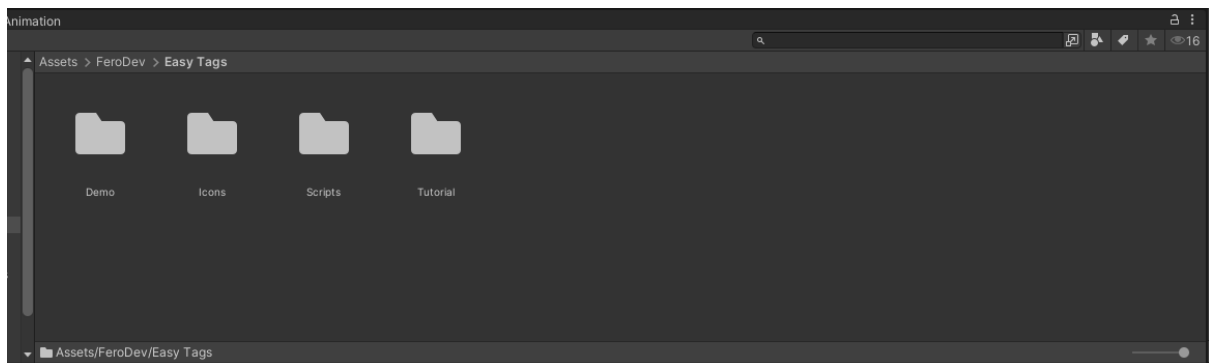
- A custom attribute for strings to display a dropdown for tags in the inspector.
- A custom property for tags to let you choose multiple tags as a mask.
- A custom component to let you assign multiple tags to your game objects.
- Accessible tags to let you access tags by their defined names instead of writing them by hand.
- A better tag management window for adding and deleting tags.
- Extensive utility functions to help you jump to development right away.
- An optimized cache-based search system for the objects that has `MultipleTags`.

Part 2: Walkthrough

When you import the package, you will see a folder with our publisher's name (FeroDev) under the Assets folder:



You will find all the source for Easy Tags under that folder:

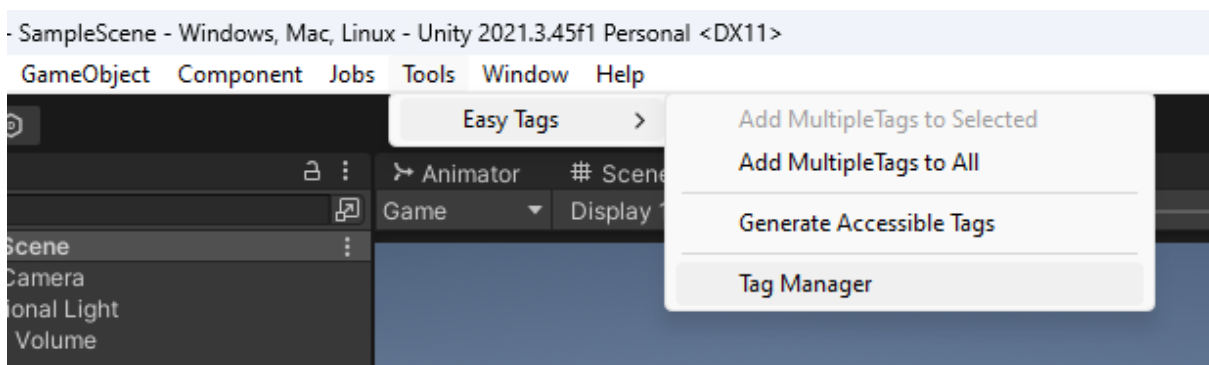


- Demo folder contains a scene to demonstrate the use of Easy Tags, however, you do not need it for any functionality, so you can delete it if you want to save space.
- Icons folder has the icons used in Easy Tags.
- Scripts folder has all the source code.
- Tutorial folder includes information about Easy Tags and how to use it. Along with Demo folder, this folder also has no direct functional files for Easy Tags.

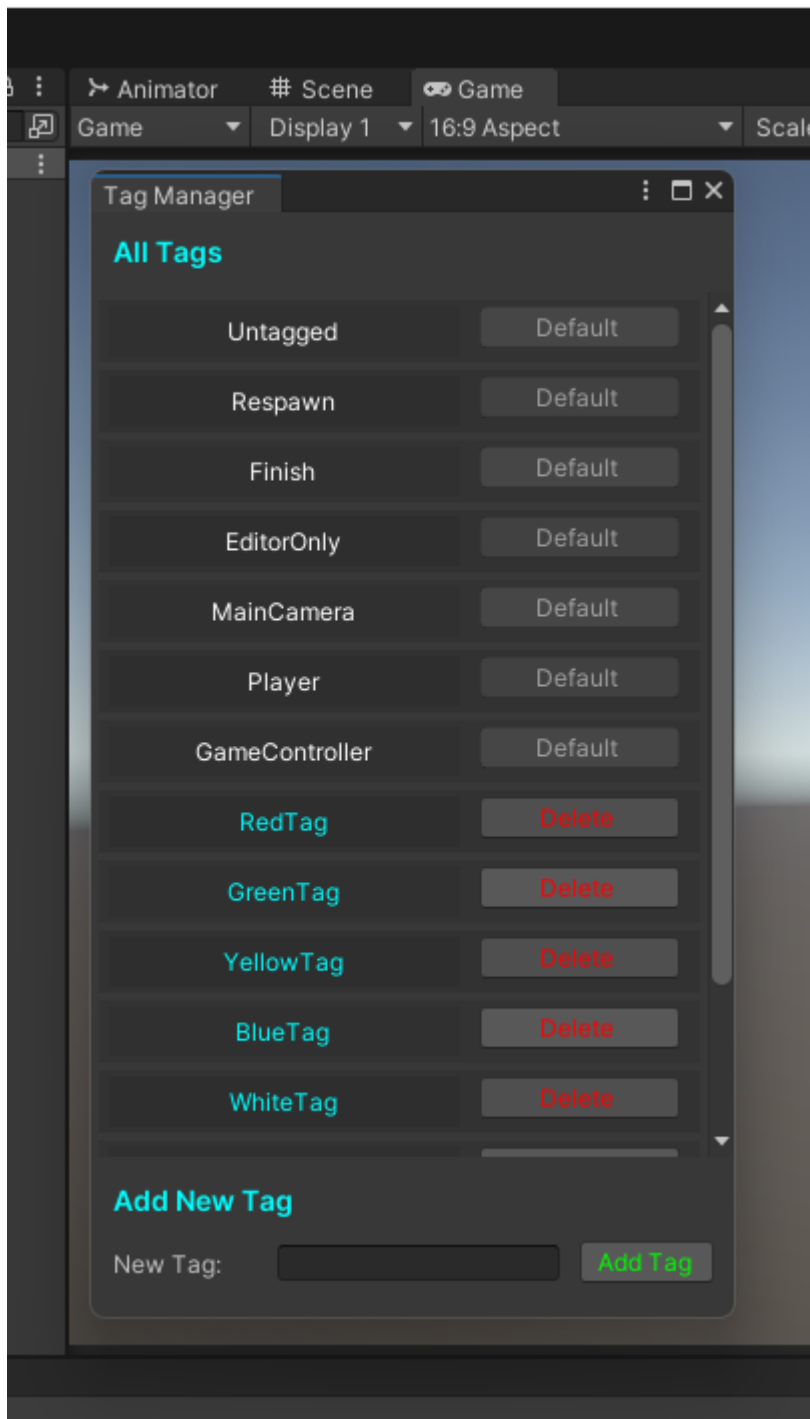
Part 3: How to use it

3.1: Tag Manager Window

To open the Tag Manager window, go to Tools -> Easy Tags -> Tag Manager

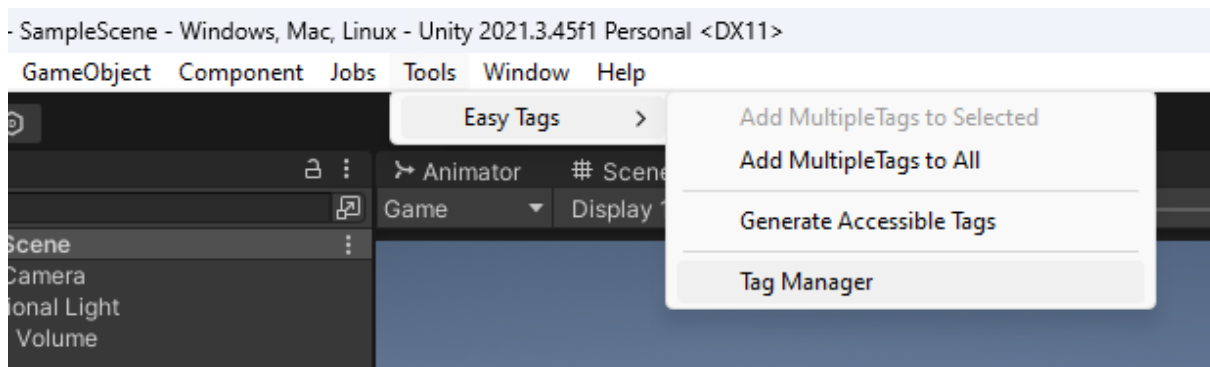


It will open a window where you can manage your tags in a more user-friendly way



3.2: Generate Accessible Tags:

Easy Tags automatically generates accessible tags, but you can also force it via the same Easy Tags menu under tools

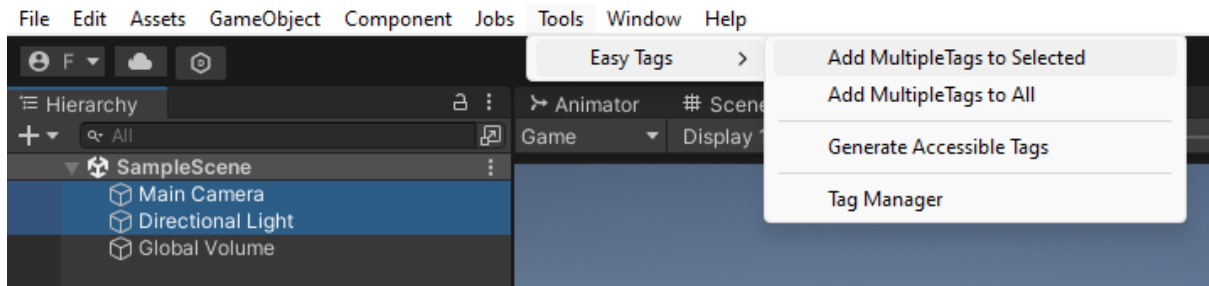


Below is a screenshot from the documentation website that shows you how you can use the accessible tags:

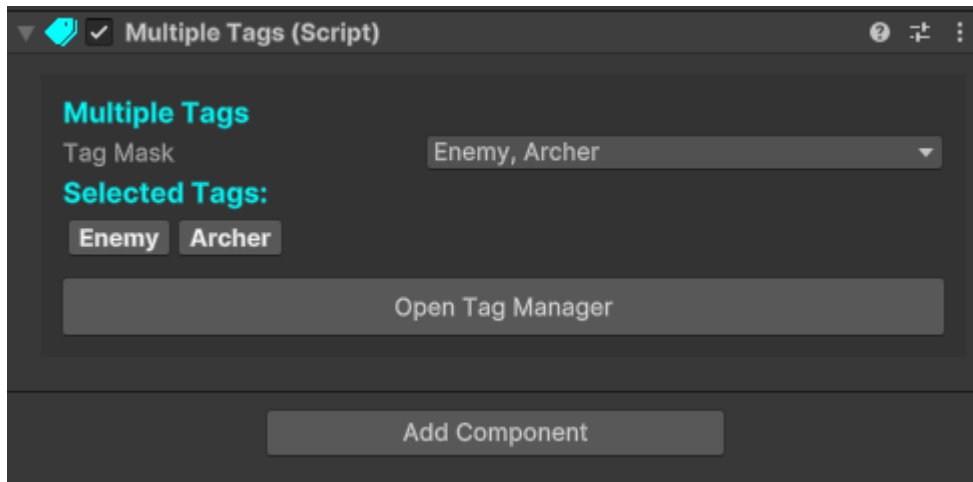


3.3: Multiple Tags Component

You can either add the Multiple Tags component using the traditional Add Component button in the inspector menu, or go to Tools -> Easy Tags -> Add Multiple Tags to Selected or Add Multiple Tags to All



It will add the following component to your game objects:



- You can add or remove multiple tags using the drop-down menu.
- You can remove tags by simply clicking on individual tags under the Selected Tags section.
- You can directly open the Tag Manager from here.

3.4: [Tag] Attribute and TagMask property

[Tag] attribute is used for strings in scripts which converts the inspector field into a dropdown menu of tags.

TagMask property is a custom property (variable) which lets you select multiple tags as a mask just like Unity's own LayerMask property.

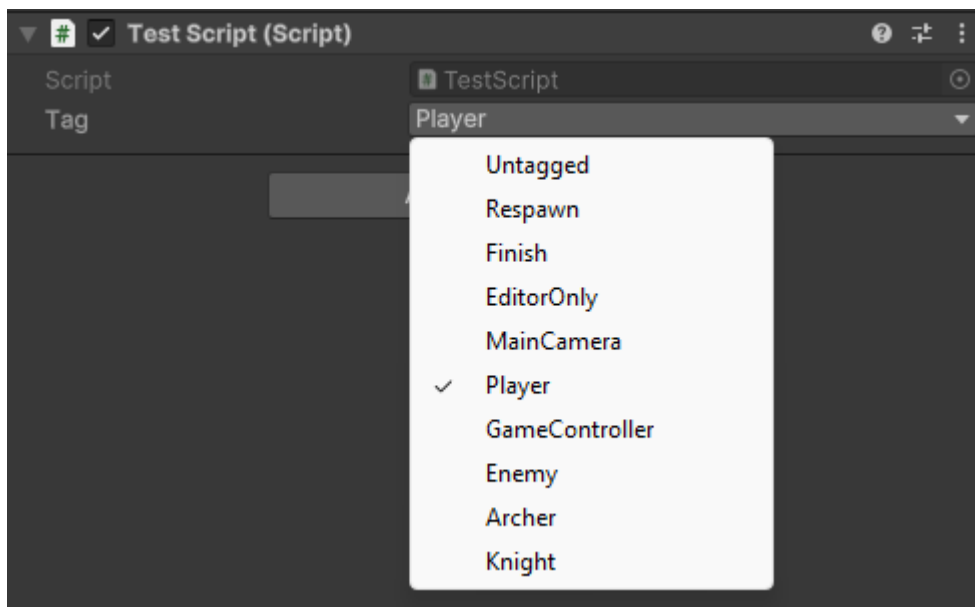
The following screenshots are taken from the documentation website:

```
// Add the namespace
using UnityEngine;
using EasyTags;

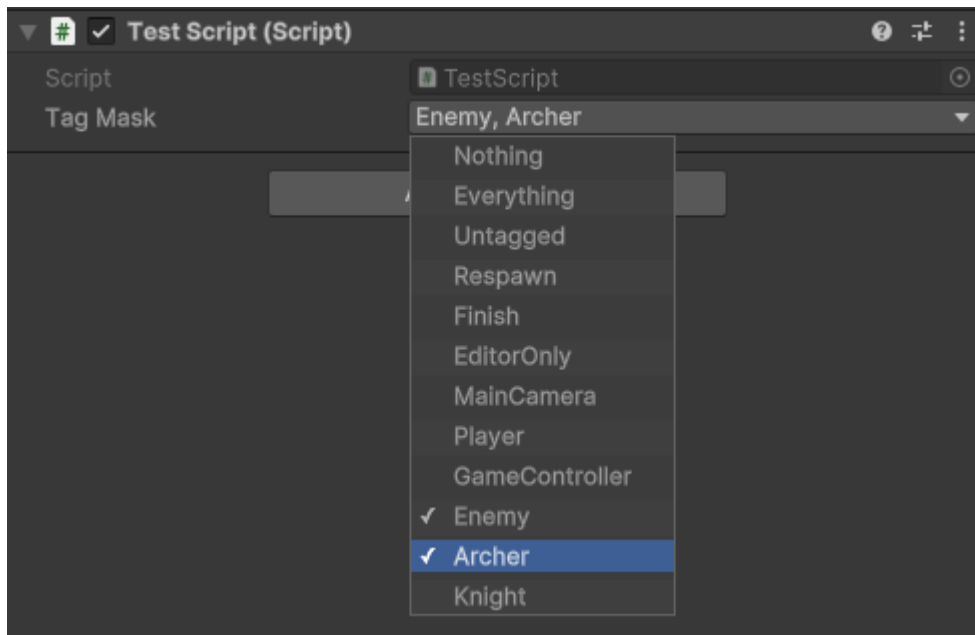
// Then you can use [Tag] attribute for tag dropdowns
[Tag] public string tag;

// And TagMask property for masks for tags
public TagMask tagMask;
```

[Tag] attribute in the inspector menu:



TagMask property in the inspector menu:



3.5: TagMask and Multiple Tags Example Usage

The following screenshot is taken from the documentation website that shows couple example usages of TagMask property and Multiple Tags functionality in code:

```
// Add the namespace
using UnityEngine;
using EasyTags;
```

Copy

```
// Adding or removing tags from a mask
public TagMask tagMask;

void Start()
{
    tagMask = new TagMask (new string[] {Tags.Enemy, Tags.Player});
    tagMask.AddTagsToMask(new string[] {Tags.Knight, Tags.Archer});
    tagMask.RemoveTagFromMask(Tags.Player);
}
```

Copy

```
// Collision handling example using MultipleTags
private void OnCollisionEnter (Collision collision)
{
    // or use MatchesWith(tagMask) based on your use case
    if (collision.gameObject.GetComponent<MultipleTags>().HasTags(tagMask))
    {
        TakeDamage(10);
    }
}
```

Copy

```
// MultipleTags uses TagMask, so something like this is also possible
string[] selectedTags = gameObject.GetComponent<MultipleTags>().TagMask.GetSelectedTags();

// Although you can directly use MultipleTags for that
string[] selectedTags = gameObject.GetComponent<MultipleTags>().GetSelectedTags();
```

Copy

3.6: TagMaskUtils Class

TagMaskUtils is a static utility class that has a lot of built-in function to use with MultipleTags and TagMask. Here is a summary of the things it can do:

- Caches game objects with MultipleTags.
- Uses the cache to efficiently search for game objects using TagMask.
- Compares game objects with TagMask.
- Checks a list or an array for similarly tagged (MultipleTags) game objects using TagMask.
- Uses the cache to efficiently search for transforms using TagMask
- Does a traditional search for traditionally tagged objects (without MultipleTags attached) using TagMask
- Modifies a TagMask using a tag or a list or array of tags.

The following screenshot is taken from the documentation website that shows couple example usages of different functions in TagMaskUtils class in code:

```
// Add the namespace
using UnityEngine;
using EasyTags;
```

Copy

```
// Prepare your TagMask either through code or in the inspector
public TagMask tagMask;

void Start()
{
    // Find specifically tagged game objects that have MultipleTags attached
    GameObject[] archerEnemies = TagMaskUtils.FindGameObjectsWithTags(tagMask);

    // Find specifically tagged transforms that have MultipleTags attached
    Transform[] archerEnemiesTransforms = TagMaskUtils.FindAllWithTags(tagMask);
}
```

Copy

```
// You can even use TagMask to search for traditionally tagged game objects
// This function finds game objects using more than one tag with TagMask
// And it works for game objects that doesn't have MultipleTags
GameObject[] allEnemies = TagMaskUtils.FindGameObjectsWithoutMultipleTags(tagMask);
```

Copy

```
// Collision handling example using TagMaskUtils
private void OnCollisionEnter(Collision collision)
{
    if (TagMaskUtils.CompareGameObjectsTag(this.gameObject, collision.gameObject))
    {
        // Do something if two MultipleTags game objects have any matching tags
    }
}
```

Copy

```
// You can also check if a list or array of GameObjects has similar MultipleTags
public GameObject[] gameObjects;

void Start()
{
    if (TagMaskUtils.HasAllMatching(gameObjects))
    {
        // Do something if all of the GameObjects have the same tags in their MultipleTags
    }
}
```

Copy